

Below are the details about how to use our lambda function to add something to the database.

First, you will need to have some way to make a POST request to this URL:

<https://tj3dwcndi76ngvuexbc6tehffi0kteha.lambda-url.us-east-1.on.aws/>

There is no security on this URL or lambda function for ease of use.

When sending the post request, the body should be formatted as shown on right. Below is the same with sample data. Notice the lack of trailing commas at the end of the lists and the end of the body. AWS does not allow trailing commas on these objects.

```
{
    "Public": ,
    "UID": "",
    "AccountID": "",
    "Recipe Name": "",
    "Directions": [],
    "Ingredient List": [],
    "TotalTime": 0,
    "ActiveTime": 0,
    "PrepTime": 0,
    "CookTime": 0,
    "Type": "",
    "Characteristics": [],
    "Description": "",
    "Summary": "",
    "VideoLink": "",
    "Links": [],
    "PairsWith": [],
    "ImageUrl": "",
    "Source": "",
    "Equipment": "",
    "Nutrition": "",
    "Servings": 0
}
```

```
{
  "Public": true,
  "UID": "00000000000000000000",
  "AccountID": "ereyasdjeaiw990",
  "Recipe Name": "Sample Recipe",
  "Directions": ["Step1", "Step2"],
  "Ingredient List": [["Ing1", 80, "cups"], ["Ing2", 3.5, "grams"]],
  "TotalTime": 90,
  "ActiveTime": 45,
  "PrepTime": 25,
  "CookTime": 20,
  "Type": "Breakfast",
  "Characteristics": ["Gluten-Free", "Vegan"],
  "Description": "This is a sample description, shown on the recipe screen",
  "Summary": "This is used as a quick look on the home screen",
  "VideoLink": "https://youtu.be/8p-f9DcVkgE",
  "Links": ["sadeaisdjasjd8asd", "osajdjkaowiisad99"],
  "PairsWith": ["Salad", "Wine"],
  "ImageUrl": "https://www.alimentarium.org/sites/default/files/media/image/2017-02/AL027-01_pomme_de_terre_0_0.jpg",
  "Source": https://www.google.com,
  "Equipment": "A bowl",
  "Nutrition": "30 carbs, 20g Fat",
  "Servings": 2
}
```

Below is the code we use to generate the body. This is written in Dart.

```
Future<String> _constructRecipeDatabaseObject(Recipe r) async {
  final String name = r.title;
  final int cookTime = r.cookTime.inMinutes;
  final int activeTime = r.activeTime.inMinutes;
  final int prepTime = r.prepTime.inMinutes;
  final int totalTime = r.totalTime.inMinutes;
  String steps = '';
  String ingredients = '';
  for (int o = 0; o < r.steps.length; o++) {
    RecipeStep s = r.steps[o];
    steps += '"${Utils.removeControlCharacters(s.description)}"$${o == r.steps.length - 1 ? '' : ','}';
  }
  for (int j = 0; j < r.ingredients.length; j++) {
    RecipeIngredient i = r.ingredients[j];
    ingredients += '"${Utils.removeControlCharacters(i.ingredient)}", $${i.amount ?? -1},
"${Utils.removeControlCharacters(i.unit)}"$${j == r.ingredients.length - 1 ? '' : ','}';
  }

  String accountId = r.ownerId;

  bool public = r.public;

  String type = r.type;
  String characteristics = '';
  for (int c = 0; c < r.characteristics.length; c++) {
    String s = r.characteristics[c];
    characteristics += '"$s"$${c == r.characteristics.length - 1 ? '' : ','}';
  }

  String recipeLinks = '';
  for (int l = 0; l < r.recipeLinks.length; l++) {
    String s = r.recipeLinks[l];
    recipeLinks += '"$s"$${l == r.recipeLinks.length - 1 ? '' : ','}';
  }

  String pairsWithList = '';
  for (int p = 0; p < r.pairsWith.length; p++) {
    String s = r.pairsWith[p];
    pairsWithList += '"${Utils.removeControlCharacters(s)}"$${p == r.pairsWith.length - 1 ? '' : ','}';
  }

  return '"{"${AppText.ingredientsKey}":[$ingredients],
"${AppText.accountIdKey}": "$accountId",
"${AppText.directionsKey}":[$steps],
"${AppText.publicKey}": $${public.toString()},
"${AppText.recipeNameKey}": "${Utils.removeControlCharacters(name)}",
"${AppText.cookTimeKey}":$cookTime,
"${AppText.prepTimeKey}":$prepTime,
"${AppText.activeTimeKey}":$activeTime,
"${AppText.totalTimeKey}":$totalTime,
"${AppText.uuidKey}": "${r.uid}",
"${AppText.typeKey}": "$type",
"${AppText.characteristicsKey}":[$characteristics],
"${AppText.descriptionKey}": "${Utils.removeControlCharacters(r.description)}",
"${AppText.summaryKey}": "${Utils.removeControlCharacters(r.summary)}",
"${AppText.videoLinkKey}": "${r.videoLink}",
"${AppText.linksKey}":[$recipeLinks],
"${AppText.pairsWithKey}":[$pairsWithList],
"${AppText.imageUrlKey}": "${Utils.removeControlCharacters(r.imageUrl)}",
"${AppText.sourceKey}": "${Utils.removeControlCharacters(r.source)}",
"${AppText.nutritionKey}": "${Utils.removeControlCharacters(r.nutrition)}",
"${AppText.equipmentKey}": "${Utils.removeControlCharacters(r.equipment)}",
"${AppText.servingsKey}": "${Utils.removeControlCharacters(r.servings.toString())}",
"Token": "${await GetIt.I<AuthService>().getAuthToken()}"
}';
}
```

To make a Post Request using Postman, get to this screen, and ensure you have the link copied in as well as the request type set to POST.

The image shows the Postman interface for creating a new POST request. At the top, there's a tab labeled "POST Hello" with a red dot indicating it's the active tab. Below this, the request type is set to "POST" and the URL is "https://15i6oogfuhemhhdpadespmfxy40xcmdg.lambda-url.us-east-1.on.aws/". The "Send" button is prominent in blue. The "Headers" tab is selected, showing a table with columns for KEY, VALUE, and DESCRIPTION. The first header is "Content-Type" with the value "application/json; charset=UTF-8". The "Body" tab is also visible, showing a "raw" radio button selected. The "Response" section is at the bottom, showing a placeholder image of a rocket and astronaut.

KEY	VALUE	DESCRIPTION
Content-Type	application/json; charset=UTF-8	
Key	Value	Description

Next, click the headers tab and enter “Content-Type” as the key and “application/json; charset=UTF-8” as the value, no quotes.

The image shows the Postman interface with the "Headers" tab selected. The "Content-Type" header is added with the value "application/json; charset=UTF-8". The "Body" tab is also visible, showing a "raw" radio button selected. The "Response" section is at the bottom, showing a placeholder image of a rocket and astronaut.

KEY	VALUE	DESCRIPTION
Content-Type	application/json; charset=UTF-8	
Key	Value	Description

Next, go to the Body Tab, and click the “raw” radio button. Ensure the drop down is JSON.

Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
Cookies
Code

none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify

1

Response

Paste in this template and adjust the data to your needs. **ENSURE THE UID FIELD IS UNIQUE AND ALL (“) CHARACTERS INSIDE TEXT BODIES ARE ESCAPED BY PUTTING A (\) IN FRONT OF THEM, LIKE THIS (“”).** Then click Send!

```
{
  "Public": true,
  "UID": "00000000000000000000",
  "AccountID": "ereyasdjeaiw990",
  "Recipe Name": "Sample Recipe",
  "Directions": ["Step1", "Step2"],
  "Ingredient List": [{"Ing1", 80, "cups"}, {"Ing2", 3.5, "grams"}],
  "TotalTime": 90,
  "ActiveTime": 45,
  "PrepTime": 25,
  "CookTime": 20,
  "Type": "Breakfast",
  "Characteristics": ["Gluten-Free", "Vegan"],
  "Description": "This is a sample description, shown on the recipe screen",
  "Summary": "This is used as a quick look on the home screen",
  "VideoLink": "https://youtu.be/8p-f9DcVkgE",
  "Links": ["sadeaisdjasjd8asd", "osajdjkaowiisad99"],
  "PairsWith": ["Salad", "Wine"],
  "ImageUrl": "https://www.alimentarium.org/sites/default/files/media/image/2017-02/AL027-01_pomme_de_terre_0_0.jpg",
  "Source": https://www.google.com,
  "Equipment": "A bowl",
  "Nutrition": "30 carbs, 20g Fat",
  "Servings": 2
}
```

Params Authorization Headers (9) **Body** Pre-request Script Tests Settings Cookies Code

● none ● form-data ● x-www-form-urlencoded ● raw ● binary ● GraphQL JSON ▼ Beautify

```
1 {
2   "Public": "true",
3   "UID": "000000000000000000",
4   "AccountID": "ereyasdjeaiw990",
5   "Recipe Name": "Sample Recipe",
6   "Directions": ["Step1", "Step2"],
7   "Ingredient List": [{"Ing1", 80, "cups"}, {"Ing2", 3.5, "grams"}],
8   "TotalTime": 90,
9   "ActiveTime": 45,
10  "PrepTime": 25,
11  "CookTime": 20,
12  "Type": "Breakfast",
13  "Characteristics": ["Gluten-Free", "Vegan"],
14  "Description": "This is a sample description, shown on the recipe screen",
15  "Summary": "This is used as a quick look on the home screen",
16  "VideoLink": "https://youtu.be/8p-f9DcVkgE",
17  "Links": ["sadeaisdjasjd8asd", "osajdjkaowiisad99"],
18  "PairsWith": ["Salad", "Wine"],
19  "ImageUrl": "https://www.alimentarium.org/sites/default/files/media/image/2017-02/AL027-01_pomme_de_terre_0_0.jpg",
20  "Source": "https://www.google.com"
21 }
```

Response

If everything works, you will see this:

Body Cookies Headers (6) Test Results Status: 200 OK Time: 305 ms Size: 265 B Save Response ▼

Pretty Raw Preview Visualize JSON ▼

```
1 "Success"
```